

Project Title

KAAVISH PROJECT PROPOSAL

By

Abdullah Shaikh (09245@st.habib.edu.pk)
Salman Kamal (sk08996@st.habib.edu.pk)
Taha Zahid (tz09220@st.habib.edu.pk)
Zain Naqi (09224@st.habib.edu.pk)

September 27, 2026



In partial fulfillment of the requirement for
Bachelor of Science
Computer Science

DHANANI SCHOOL OF SCIENCE AND ENGINEERING

HABIB UNIVERSITY

FALL 2023

Copyright © 2023 Habib University

1 Abstract

Robots are increasingly used to perform complex tasks in warehouses, hospitals, construction sites, and disaster response scenarios. In many of these situations, a single robot is not enough. A task might be too large, require different types of equipment, or need to be finished quickly. This is where multi-robot systems come in, where teams of robots work together toward a shared goal. Coordinating that team, though, is not simple. Deciding which robot should do which task, in what order, and by when is a hard problem in itself. This problem is called Multi-Robot Task Allocation, or MRTA.

For a long time, MRTA systems were programmed for specific scenarios. A team of robots built to sort packages in a warehouse could do exactly that, but nothing more. If the mission changed or an unexpected situation came up, the system had no way to handle it. The robots could only do what they were explicitly programmed for.

Large Language Models changed what was possible here. These models, trained on large amounts of text, can read natural language instructions, reason about goals, and break a complex task into smaller steps. Researchers started using them in multi-robot systems so that a human could describe a mission in plain language and have the robots figure out how to handle it. This opened up a lot of new possibilities. The problem is that most of these systems also use the LLM to decide which robot gets which task. That step, deciding who does what, is a mathematical problem. It requires finding an assignment that satisfies constraints around robot capabilities, travel distances, timing, and whether some tasks need to be done simultaneously by multiple robots. LLMs were not built for this and do not handle it well. Research has shown that when asked to make these decisions, they tend to produce assignments that are unbalanced, infeasible, or far from optimal, even when given the chance to review and correct their own output.

This project takes a different approach. The LLM is used only for the parts of the problem it handles well: reading the mission description, breaking it into subtasks, figuring out which subtasks depend on which others, and determining what type of coordination problem the mission represents. Once the problem type is identified, a dedicated mathematical solver handles the actual assignment of robots to tasks. Different types of problems have different mathematical structures, and there are well-established algorithms for each. Using the right solver for each problem type produces better results than asking the LLM to figure everything out.

We test this system in a simulated environment using a mixed fleet of wheeled ground robots, robotic arms, and aerial drones, all with different capabilities. The simulation covers seven different problem types, from straightforward cases where robots independently handle separate tasks to more involved scenarios where groups of robots need to coordinate in a particular order. We compare the results against systems that leave the allocation step to the LLM, using measures like total mission time, travel distance, and how far the solution is from the best possible assignment.

The aim is to show that keeping task allocation as a mathematical problem, and using the LLM only for language understanding, leads to more dependable outcomes. If this holds up, it could make multi-robot systems more practical in settings where the missions vary and cannot all be anticipated in advance.

2 Problem definition

Multi-Robot Task Allocation (MRTA) is the problem of computing an assignment from a set of tasks $\mathcal{T}$ to a team of heterogeneous robots $\mathcal{R}$ such that some global objective, typically total

utility or mission makespan, is optimized subject to physical and operational constraints [1]. The computational difficulty of this problem is not uniform across all scenarios; it depends on the structure of the robots, the tasks, and the dependencies among them.

Gerkey and Mataric [1] formalized this through a three-axis taxonomy. The first axis distinguishes Single-Task robots (ST), which execute at most one task at a time, from Multi-Task robots (MT). The second axis distinguishes Single-Robot tasks (SR), requiring exactly one robot, from Multi-Robot tasks (MR), requiring a coalition of robots with complementary capabilities. The third axis distinguishes Instantaneous Assignment (IA), where the assignment is made once based on information available at that moment with no planning for future tasks, from Time-Extended Assignment (TA), where a full schedule is constructed over a planning horizon. These axes yield distinct optimization problems. ST-SR-IA reduces to the Linear Sum Assignment Problem (LSAP), solvable in $O(n^3)$ via the Hungarian algorithm. ST-MR-IA corresponds to the Set Partitioning Problem (SPP). ST-SR-TA corresponds to scheduling on unrelated parallel machines ($R|| \sum w_j C_j$). The latter two are strongly NP-hard.

Korsah et al. [2] identified a significant gap in this taxonomy: it assumes that the utility of assigning any robot to any task is independent of all other assignments. This assumption fails in most real-world missions. Their extended taxonomy, iTax, introduces a dependency dimension. In No-Dependency (ND) problems, utilities are fully decoupled and the problem reduces to a variant of LSAP. In In-Schedule Dependency (ID) problems, a robot's cost for a task depends on the sequence of its own assigned tasks, as in the Multiple Traveling Salesperson Problem (m-TSP) or the Capacitated Vehicle Routing Problem (CVRP). In Cross-Schedule Dependency (XD) problems, the schedules of different robots are coupled through inter-task constraints such as precedence, temporal synchronization, or coalition requirements. XD problems require joint schedule optimization across robots and map to formulations including the Vehicle Routing Problem with Time Windows and Precedence Constraints (VRPTW), disjoint coalition formation via Set Partitioning, and the Coalition Formation with Spatial and Temporal Constraints (CFSTP). This project targets seven subclasses spanning these tiers: ND[ST-SR-IA], ND[ST-SR-TA], ID[ST-SR-TA], XD[ST-SR-IA], XD[ST-SR-TA], XD[ST-MR-IA], and XD[ST-MR-TA].

A related challenge is the interface between human operators and these systems. Traditional MRTA systems require missions to be specified as structured inputs, which limits them to the tasks they were explicitly programmed for. Large Language Models (LLMs) address this by enabling systems to accept missions described in arbitrary natural language, making the system generalizable across problem domains without requiring task-specific programming. Li et al. [3] survey how this has been applied in multi-robot systems and find a consistent pattern: most systems do not limit the LLM to mission interpretation and decomposition; they use it to perform task allocation as well, deciding which robot does what. This is problematic because allocation, regardless of which subclass applies, requires combinatorial reasoning over numerical constraints. LLMs consistently produce suboptimal, infeasible, or poorly balanced allocations under these conditions, even when given access to self-correction mechanisms [3].

This project addresses two related problems. First, how can a natural language mission be interpreted and decomposed into a structured set of subtasks with dependencies, and correctly classified into one of the iTax subclasses? Second, given that classification, which optimization algorithm should be applied, and how should its output be dispatched to a heterogeneous robot fleet? The key observation is that these two sub-problems require different computational tools. Language understanding and combinatorial optimization are not interchangeable, and systems that use an LLM for both are neither formally sound nor practically reliable.

3 Social relevance

Multi-robot systems can support work that is important to people. In hospitals, they can move medicine and equipment. In disasters, they can enter unsafe areas to search for survivors or deliver supplies. In warehouses and farms, they can help move goods and monitor large areas. These uses can save time, reduce physical workload, and keep people away from dangerous tasks.

For these benefits to be useful, the robots must be given suitable work and must work together safely. A poor decision can delay urgent supplies, leave an important task unfinished, or send the wrong robot to a job. This project supports more reliable coordination of robot teams. In the long term, this can help organisations use robots more safely and effectively in services that affect the public.

4 Originality/Novelty

The classical approach to MRTA relies on hand-crafted allocation mechanisms such as market-based auctions, greedy matchers, or fixed optimisation formulations. Gerkey and Mataric [1] showed that architectures like ALLIANCE, BLE, and MURDOCH are all variants of the same greedy algorithm applied to the Optimal Assignment Problem, and that this greedy approach achieves at most a 2-competitive or 3-competitive ratio relative to the optimal solution. These systems are mathematically well-studied and produce predictable results, but they require missions to be specified as structured inputs and are designed around a fixed allocation method. They do not generalise across problem types, and they cannot interpret a mission described in natural language.

LLM-based multi-robot systems address the natural language limitation but introduce a different problem. As documented by Li et al. [3], the dominant pattern in this literature is to use the LLM as a general-purpose coordinator that handles both mission interpretation and task allocation. This works well for the interpretation step but breaks down at allocation, since deciding which robot does which task under capability, precedence, and resource constraints is a combinatorial optimisation problem. LLMs are not combinatorial solvers, and studies show they produce infeasible or suboptimal allocations even with self-correction mechanisms in place [3]. The failure is not incidental; it reflects a fundamental mismatch between what LLMs are designed to do and what allocation requires.

Some recent work has moved toward hybrid architectures that pair an LLM with a downstream solver. LiP-LLM uses an LLM to extract task structure and then solves a Linear Program for assignment. Peng et al. use an LLM to generate MILP code for a specific manufacturing domain. DART-LLM parses natural language into a task DAG and dispatches to navigation stacks [3]. These systems are a step in the right direction, but they are each built around a single optimisation model suited to a specific domain. None of them uses a taxonomy to classify the problem type before selecting a solver. As a result, they cannot handle the structural diversity of MRTA problems: a system built around a linear assignment model has no mechanism for coalition formation, and one built around a vehicle routing formulation cannot handle instantaneous single-assignment problems correctly.

This project's contribution is the taxonomy classification layer that sits between the LLM and the optimisation engine. The LLM parses the natural language mission and classifies it into one of seven iTax subclasses [2], each corresponding to a distinct mathematical formulation. The system then routes to a solver specifically designed for that formulation, ranging from the Hungarian algorithm for ND[ST-SR-IA] to MILP-based coalition scheduling for XD[ST-MR-TA]. This is the

first system, to our knowledge, that uses LLM-based taxonomy classification as a principled routing mechanism across the full ND-ID-XD spectrum of MRTA problems. The result is a pipeline where natural language generalizability and mathematical rigour are achieved together rather than traded off against each other.

5 CS contribution

This project requires knowledge from several areas of Computer Science. The first area is artificial intelligence and natural language processing. We will build an LLM-based planning component with retrieval-augmented generation (RAG). It will retrieve the robot capability library, map information, and task-allocation taxonomy, and use this context to extract subtasks, target locations, required capabilities, estimated durations, and task dependencies from a natural-language mission. The component will produce a schema-constrained JSON plan that is parsed and validated by program code before it can be used by later modules. This requires work on prompt design, retrieval and indexing, context selection, structured output generation, and error handling for incomplete or invalid plans.

The second area is algorithms and graph-based modelling. The mission will be represented as a Directed Acyclic Graph (DAG), where each node is a subtask and each edge shows that one task must be completed before another. We will validate the graph and use its dependency information when planning the task order. This applies core concepts from data structures and graph algorithms to a practical robotics problem.

The main technical contribution is combinatorial optimisation. Each type of task-allocation problem will be mapped to a suitable mathematical model, such as an assignment problem, a routing problem, or a coalition-formation problem. Simple cases can be solved exactly using methods such as the Hungarian algorithm. More difficult cases, including variants of the Travelling Salesperson Problem, require integer programming or heuristic methods because they are computationally hard. This work draws directly on the optimisation and heuristic-search concepts studied in CS 451: Computational Intelligence.

Finally, the project includes robotics software integration and experimental evaluation. We will implement ROS nodes that convert the selected schedule into robot-specific action commands. These nodes will call the navigation, manipulation, or control interfaces needed for each subtask, monitor whether commands have been accepted or completed, and coordinate the order in which robots receive their work. The LLM provides the high-level mission plan; the ROS control code and robot controllers carry out movement and task execution in Gazebo. We will then measure mission time, travel distance, computation time, and solution quality to compare the system with baseline approaches under the same simulated conditions.

6 Scope and Deliverables

The project will be completed over one academic year in five stages. Its scope is limited to a mapped, simulated environment with reliable communication and open-loop execution. The project will not develop new low-level navigation, manipulation, or flight controllers. Instead, it will send high-level tasks to existing robot-control interfaces in ROS and Gazebo. This boundary keeps the work focused on task planning, task allocation, integration, and evaluation.

The following are the planned deliverables:

1. **Environment plan, 2D test representation, and mission set:** By the end of the first stage, we will define the robot fleet, task stations, map layout, and task requirements for the planned environment. We will prepare at least one standard natural-language mission for each of the seven selected iTax problem classes and represent these missions in a 2D test setting for early development.
2. **Mission-planning and classification module:** By the end of the second stage, we will implement an LLM and RAG module that converts each test mission into a validated JSON task plan. The plan will include subtasks, capabilities, locations, and dependencies represented as a DAG. The module will also assign one of the seven target taxonomy classes to each mission. This module will initially be tested in the 2D setting, with limited ROS or Gazebo checks where needed.
3. **Task-allocation solvers:** In parallel with the planning module, we will implement or adapt one suitable solver for each of the seven problem classes. For every standard mission, the solver must return a feasible robot assignment and schedule that satisfies the stated capability, dependency, and timing constraints.
4. **Integrated ROS–Gazebo pipeline:** By the fourth stage, we will build the 3D Gazebo environment and connect the validated mission plan, solver-routing layer, and ROS execution nodes. The complete pipeline will accept a test mission, select the appropriate solver, create a schedule, and dispatch the resulting actions to the simulated robots.
5. **Benchmark and evaluation report:** In the final stage, we will run the full pipeline on the standard mission set and compare it with selected baseline approaches, including direct LLM planning and a greedy allocation method where applicable. We will report classification accuracy, feasibility of the produced schedules, mission completion time, total travel distance, solver time, and optimality gap when an optimal solution is available.

The first stage will focus on environment planning, a 2D test representation, and the mission set. The planning module and optimisation solvers will be developed and tested in parallel during the second stage. The 3D Gazebo environment and full ROS integration will be completed in the fourth stage, followed by simulation execution and benchmarking. This order allows each component to be tested on its own before the complete system is evaluated.

7 Feasibility

List the resources, e.g. datasets, compute resources, software libraries, hardware, required for the project. Mention how you expect to access and utilize them for the project.

8 Team dynamics

The team has four members with skills that match the main parts of the project. Responsibilities are divided clearly, while members will work together during integration and testing.

Abdullah will lead ROS and Gazebo integration for the LLM-controlled robot system. His STRP work gave him experience with LLM-controlled multi-robot systems. His autonomous tow-truck

project at Toyota gave him experience with ROS and Gazebo. He is also the only member with prior experience of RAG pipelines and Gazebo.

Salman will lead the design of the simulation environment and contribute to evolutionary optimisation methods. His game-development experience is useful for building the simulation environment, while his AI coursework supports his work on heuristic methods.

Zain and Taha will map taxonomy classes to optimisation models and implement the task-allocation solvers. They have the strongest background in Computational Intelligence and optimisation algorithms. Their work will cover assignment, routing, scheduling, and coalition-formation problems.

All members have completed or are completing Mobile Robotics, giving the team a shared understanding of ROS and multi-robot systems. Abdullah's RAG and Gazebo experience, Salman's design skills, and Zain and Taha's stronger optimisation background cover the main areas in which the other team members have less experience.

9 Tech stack

Write details of the tech stack you will use for this project for e.g. if you are using MERN stack, you can write MongoDB, Express, React and NodeJS etc.

10 References

References

- [1] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004.
- [2] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research*, 32(12):1495–1512, 2013.
- [3] Peihan Li, Zijian An, Shams Abrar, and Lifeng Zhou. Large language models for multi-robot systems: A survey, 2026.

Undertaking of Kaavish advisement as an External Supervisor

I hereby affirm that I have read the project details as described on the preceding pages and agree to undertake advisement of this Kaavish project as an External Supervisor. I understand that this role entails the following.

Meeting Meeting the project team regularly, at least once every two weeks, for the entire duration of the Kaavish. The meetings may be held remotely if required.

Advisement Providing supervision and advice to the team in order to ensure steady progress of the project toward its goals.

Liaison Liaising with the Internal Supervisor as required, e.g. to provide feedback or engage in grading.

Other Any other task, depending on availability and suitability, relevant to the Kaavish as communicated by the Internal Supervisor or Kaavish Working Group.

Name: _____

Email: _____

Phone: _____

Designation: _____

Affiliation: _____

Signature: _____